

ВВЕДЕНИЕ

Современный этап развития мировых социально-экономических отношений характеризуется беспрецедентным уровнем глобализации и устойчивым ростом интенсивности международных перемещений. Несмотря на периодические колебания, вызванные глобальными кризисами, общая тенденция к расширению географии деловых поездок и индивидуального туризма сохраняет свою актуальность. В этих условиях обеспечение безопасности путешественников становится одной из приоритетных задач как для корпоративного сектора, так и для специализированных туристических компаний. Развитие концепции «должной осмотрительности» (Duty of Care), закрепленной в международных стандартах, таких как ISO 31030, возлагает на организации юридическую и моральную ответственность за жизнь и здоровье своих клиентов и сотрудников во время их нахождения за рубежом.

Ландшафт глобальных угроз в последние годы претерпел существенные изменения, став значительно более динамичным, многофакторным и труднопредсказуемым. Путешественники ежедневно сталкиваются с широким спектром рисков: от внезапных геополитических конфликтов, массовых протестов и террористических актов до стремительно развивающихся природных катаклизмов, вызванных климатическими изменениями. Эпидемиологические угрозы, сбои в работе критической транспортной инфраструктуры и возрастающий уровень локальной криминогенной напряженности требуют непрерывного контроля. В такой нестабильной среде традиционные методы оценки рисков, опирающиеся на статические страновые отчеты и периодические сводки, демонстрируют свою полную несостоятельность.

Анализ достижений в области управления рисками в путешествиях показывает, что существующие на рынке решения исторически развивались по двум основным направлениям. Первое направление базируется на экспертно-аналитическом подходе, при котором штат специалистов вручную осуществляет мониторинг новостных лент, верификацию информации и рассылку предупреждений. Данный метод обеспечивает высокую точность и отсутствие ложных срабатываний, однако характеризуется критически низкой скоростью реакции (задержки могут достигать нескольких часов) и колоссальными операционными издержками, что делает такие системы недоступными для малого бизнеса и индивидуальных туристов. Второе направление использует алгоритмический сбор данных на основе жестко заданных ключевых слов. Несмотря на высокую скорость обработки информации, подобные агрегаторы генерируют неприемлемый уровень информационного шума, поскольку алгоритмы не способны улавливать контекст, тональность сообщения и отличать реальную угрозу от исторической справки или метафоры в тексте новостной статьи.

Настоящим прорывом в решении проблемы глобального мониторин-

га стало стремительное развитие технологий искусственного интеллекта, в частности, методов обработки естественного языка (Natural Language Processing, NLP) и больших языковых моделей (Large Language Models, LLM). Появление архитектур на базе трансформеров позволило перейти от простого синтаксического поиска к глубокому семантическому анализу неструктурированных текстовых данных. Использование методов Zero-shot классификации дает возможность сопоставлять поступающие новостные сообщения с обширной таксономией угроз без необходимости постоянного переобучения моделей на новых наборах данных. Интеграция алгоритмов продвинутого геопарсинга позволяет с высокой точностью извлекать из текста пространственные координаты описываемых событий, формируя зоны потенциального поражения. Совокупность этих инновационных подходов открыла возможность создания систем мониторинга, способных обрабатывать огромные массивы информации из систем открытой разведки (OSINT), таких как проект GDELT, в режиме реального времени, извлекая только релевантные данные и сводя к минимуму участие человека в процессе первичной фильтрации.

Целью дипломного проектирования является разработка интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников по индивидуальному заказу туристической компании ООО «ТревелСейф».

Реализация данной цели направлена на автоматизацию процесса сбора, семантического анализа и оперативного оповещения об угрозах для обеспечения принципа «должной осмотрительности» и надежной защиты клиентов компании во время их поездок.

В основу проектирования, научного исследования и поиска технического решения положены следующие принципы:

- использование событийно-ориентированной микросервисной архитектуры для обеспечения высокой отказоустойчивости и сверхвысокой скорости обработки независимых потоков данных;
- применение передовых моделей машинного обучения для глубокого семантического анализа открытых источников без необходимости ручной модерации входящего контента;
- внедрение алгоритмов точного геопарсинга для автоматического извлечения пространственных координат инцидентов из неструктурированных текстовых массивов;
- интеграция механизмов геофенсинга для непрерывного сопоставления текущего местоположения пользователя или его планируемого маршрута с динамически изменяющимися зонами риска;
- применение локальных больших языковых моделей для автоматического реферирования новостных сводок и генерации лаконичных уведомлений;

- изоляция промежуточных сырых данных от очищенной аналитической информации на уровне структуры базы данных для обеспечения чистоты анализа;

- ориентация на конечного пользователя посредством создания интуитивно понятного интерфейса с интерактивной глобальной картой и гибкой системой настройки профиля угроз.

Для достижения поставленной цели в рамках дипломного проектирования необходимо решить следующие задачи:

- провести комплексный анализ предметной области и существующих на мировом рынке систем управления рисками в путешествиях;

- выявить недостатки текущих аналогов и обосновать необходимость применения интеллектуального подхода на базе нейронных сетей;

- выполнить сравнительный анализ источников данных о глобальных инцидентах и выбрать наиболее оптимальные агрегаторы;

- обосновать выбор архитектурного паттерна, технологического стека и конкретных моделей машинного обучения для реализации вычислительного ядра системы;

- спроектировать микросервисную архитектуру программного комплекса и разработать схемы взаимодействия его внутренних компонентов;

- разработать структуру реляционной базы данных с поддержкой пространственных запросов для хранения информации об инцидентах и пользователях;

- рассчитать экономические показатели целесообразности инвестиций в разработку и внедрение программного средства для организации заказчика.

Пояснительная записка к дипломному проекту структурно состоит из введения, трех основных разделов и заключения.

Краткое изложение содержания разделов пояснительной записки:

- в первом разделе проводится анализ подходов к разработке интеллектуальной системы мониторинга рисков, исследуются существующие на рынке аналогичные решения, подробно рассматриваются источники данных о глобальных событиях, а также обосновывается выбор технологий, фреймворков и моделей машинного обучения;

- во втором разделе приведено описание функциональных требований к системе, выполнено архитектурное проектирование ее микросервисных компонентов, а также представлено детальное описание структуры базы данных, программных интерфейсов и алгоритмов взаимодействия модулей сбора, анализа и оповещения;

- в третьем разделе осуществляется экономическое обоснование целесообразности инвестиций в разработку программного средства, включающее расчет затрат на заработную плату разработчиков, формирование отпускной цены системы, а также расчет показателей экономической эффективности,

таких как чистый дисконтированный доход и срок окупаемости;

– в заключении подводятся итоги проделанной работы, формулируются основные выводы по результатам проектирования и экономического обоснования, а также подтверждается успешное достижение поставленной цели дипломного проекта.

Дипломный проект выполнен самостоятельно, проверен в системе "Антиплагиат". Процент оригинальности соответствует норме, установленной кафедрой ИИТ. Цитирования обозначены ссылками на публикации, указанные в "Списке использованных источников".

1 АНАЛИЗ ПОДХОДОВ К РАЗРАБОТКЕ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ МОНИТОРИНГА РИСКОВ И ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПУТЕШЕСТВЕННИКОВ

В данном разделе проводится глубокий и всесторонний анализ сферы управления рисками в путешествиях, исследуются существующие на рынке аналогичные решения и выявляются их недостатки. Также подробно рассматриваются источники данных о глобальных событиях, проводится сравнительный анализ архитектурных подходов и обосновывается выбор конкретных технологий, фреймворков и моделей машинного обучения, необходимых для реализации интеллектуальной системы мониторинга и оповещения.

1.1 Анализ предметной области

Системы управления рисками в путешествиях представляют собой комплексные программно-аппаратные и организационные решения, предназначенные для заблаговременного выявления, оценки, мониторинга и минимизации угроз, с которыми могут столкнуться сотрудники корпораций или частные путешественники. Развитие глобализации и увеличение объемов международных перемещений привели к тому, что обеспечение безопасности поездок стало одной из ключевых задач корпоративного управления.

Современная концепция управления рисками в путешествиях базируется на принципе «должной осмотрительности», согласно которой организация несет юридическую и моральную ответственность за безопасность своих сотрудников, находящихся в командировках. В 2021 году был опубликован международный стандарт ISO 31030 «Управление рисками в путешествиях — руководство для организаций»[1], который систематизировал подходы к оценке рисков и установил высокие требования к системам мониторинга. Данный стандарт подчеркивает необходимость не только реактивного реагирования на инциденты, но и проактивного анализа потенциальных угроз еще на этапе планирования поездки. Экономические последствия пренебрежения данными стандартами могут быть катастрофическими для бизнеса. Они включают в себя не только прямые финансовые потери от необходимости экстренной медицинской эвакуации, репатриации или отмены сложных маршрутов (стоимость которых может достигать сотен тысяч долларов), но и масштабные репутационные издержки, а также многомиллионные судебные иски со стороны пострадавших сотрудников. Кроме того,

современные исследования в области организационной психологии показывают, что отсутствие видимой, понятной и надежной системы обеспечения безопасности критически снижает лояльность персонала и эффективность выполнения командировочных задач из-за повышенного уровня фоновogo стресса в незнакомой среде. В связи с этим парадигма корпоративной безопасности окончательно сместилась от пассивного страхования рисков постфактум к активному технологическому мониторингу и превентивному управлению маршрутами на основе данных объективного контроля.

Ландшафт глобальных угроз в середине 2020-х годов характеризуется высочайшей динамичностью, непредсказуемостью и каскадным эффектом, когда природные катаклизмы мгновенно провоцируют техногенные аварии и социальные волнения. В рамках исследования предметной области была разработана расширенная таксономия рисков: **Геополитические и социально-политические риски:**

- вооруженные конфликты, локальные боестолкновения и пограничные инциденты;
- государственные перевороты, резкая смена политических режимов и гражданское неповиновение;
- массовые протесты, забастовки работников транспортной отрасли и гражданские беспорядки;
- террористические акты, угрозы экстремистской деятельности и диверсии на объектах инфраструктуры;
- дипломатические инциденты, изменения в визовой политике и введение санкционных ограничений.

Природные и климатические риски:

- сейсмическая активность, включающая землетрясения различной магнитуды и угрозы возникновения цунами;
- метеорологические экстремумы, такие как ураганы, тайфуны, наводнения и аномальные температурные волны;
- лесные пожары, извержения вулканов и пылевые бури, ограничивающие видимость и работу авиации;
- долгосрочные климатические изменения, влияющие на устойчивость инфраструктуры конкретного региона.

Эпидемиологические и медицинские угрозы:

- вспышки инфекционных заболеваний, локальные очаги эпидемий и глобальные пандемии;
- низкий уровень развития местной системы здравоохранения и дефицит квалифицированного персонала;
- отсутствие доступа к специфическим медикаментам или качественной скорой помощи;
- специфические санитарно-гигиенические риски, связанные с качеством воды и пищевых продуктов.

Криминальные и кибернетические угрозы:

- уличная преступность, нападения с целью грабежа и мошенничество в отношении туристов;
- похищения людей с целью выкупа или политического давления;
- кибершпионаж через публичные незащищенные сети связи и фишинговые атаки;
- кража персональных данных, документов и физических носителей информации у путешественников.

Техногенные и инфраструктурные риски:

- аварии на объектах критической инфраструктуры, включая энергетические и водопроводные сети;
- масштабные сбои в работе транспортных узлов, аэропортов и железнодорожных вокзалов;
- энергетические кризисы, приводящие к веерным отключениям электричества и связи.

В ходе проведенного исследования установлено, что эффективность любой системы мониторинга определяется «индексом своевременности оповещения» — временным интервалом между возникновением инцидента и получением уведомления конечным пользователем. Для глобальных систем этот показатель должен стремиться к значениям менее 15 минут, что невозможно реализовать без глубокой автоматизации процессов сбора и анализа данных. Сфера управления рисками в путешествиях трансформировалась из узкоспециализированной области безопасности в критически важный элемент корпоративного управления и личной безопасности. Современные стандарты, такие как ISO 31030[1], диктуют необходимость создания систем, способных обрабатывать огромные потоки информации в режиме реального времени. Выявленная таксономия рисков демонстрирует многогранность угроз, что требует применения междисциплинарного подхода при проектировании архитектуры интеллектуальной системы.

1.2 Анализ сценариев использования и пользовательских ожиданий

С точки зрения конечного пользователя ценность системы определяется не только точностью выявления угроз, но и тем, насколько быстро, понятно и в подходящем канале пользователь получает информацию. Путешественник ожидает, что система будет работать как фоновый защитный контур: заранее предупреждать о рисках при планировании маршрута, отображать актуальные события на карте, позволять уточнять картину по категориям угроз и доставлять критические сообщения даже в ситуациях, когда пользователь не находится в веб-интерфейсе.

В рамках анализа выделены ключевые сценарии использования, ко-

торые должны быть отражены как в функциональной модели, так и в пользовательском интерфейсе системы:

- регистрация и авторизация пользователя должны обеспечивать безопасный доступ к персональным маршрутам, настройкам уведомлений и истории взаимодействия с системой;
- мониторинг инцидентов и событий безопасности должен быть доступен как обычным пользователям (для оценки рисков в рамках планирования личных поездок), так и менеджерам по управлению рисками для глобального контроля обстановки и обеспечения безопасности сотрудников;
- просмотр карты рисков должен предоставлять пользователям и менеджерам оперативное представление о текущей обстановке в выбранном регионе с визуальным выделением зон влияния инцидентов;
- фильтрация событий по типу угрозы, уровню критичности и времени публикации должна помогать быстро отделять значимые риски от фонового информационного шума в зависимости от задач конкретной роли;
- настройка маршрута поездки должна позволять системе сопоставлять планируемые перемещения пользователя с геометрией активных зон риска и выявлять потенциальные пересечения;
- получение уведомлений о безопасности должно включать как мгновенные сообщения в интерфейсе, так и оповещения по электронной почте для случаев, когда приложение закрыто;
- получение почтовых уведомлений должно обеспечивать доставку структурированного сообщения с названием события, уровнем угрозы, кратким описанием, регионом риска и практической рекомендацией;
- мониторинг состояния сервисов должен давать администратору возможность контролировать работоспособность конвейера сбора, анализа, хранения и доставки уведомлений.

Отдельного внимания требует сценарий получения почтовых уведомлений, так как он является резервным и юридически значимым каналом коммуникации. В отличие от мгновенного уведомления, которое может быть ограничено настройками мобильной операционной системы, сообщение электронной почты сохраняется в почтовом ящике пользователя, может быть повторно открыто позднее и подходит для документирования факта информирования в рамках принципа должной осмотрительности. Поэтому система должна формировать письма автоматически, на языке, выбранном пользователем, с минимальным количеством лишней информации и с четкой структурой, позволяющей быстро оценить угрозу даже в стрессовой ситуации.

1.3 Анализ подходов к решению задачи

При разработке систем мониторинга безопасности путешественников выделяют три фундаментальных подхода к сбору и обработке данных, каждый из которых имеет свои преимущества и ограничения.

Первый подход — экспертно-аналитический. Он основан на работе штатных аналитиков, которые вручную отбирают новости из проверенных источников, проводят верификацию данных через собственные каналы и формируют аналитические отчеты. Преимущества данного подхода:

- высочайшая точность, глубина контекстуального анализа и достоверность предоставляемых сведений;
- практически полное отсутствие ложных срабатываний, вызванных метафоричностью или иронией в текстах;
- возможность составления долгосрочного прогноза развития ситуации на основе инсайдерской информации.

Недостатки:

- критически низкая скорость реакции на динамические инциденты из-за человеческого фактора;
- чрезвычайно высокая стоимость содержания круглосуточного штата высококвалифицированных аналитиков;
- ограниченность масштабирования, не позволяющая одновременно отслеживать тысячи локальных источников на разных языках.

Второй подход — алгоритмический сбор на основе ключевых слов и регулярных выражений. Системы настраиваются на автоматический поиск определенных комбинаций слов в новостных лентах и социальных сетях. Преимущества:

- высокая скорость первичной обработки данных и возможность охвата большого количества источников;
- относительно низкая стоимость реализации и простота технической поддержки алгоритмов.

Недостатки:

- критически высокий уровень информационного шума и ложных срабатываний на нерелевантные новости;
- значительные трудности с автоматическим определением точной геолокации и границ зоны инцидента;
- неспособность алгоритмов понимать сложный контекст, тональность сообщения и выявлять дезинформацию.

Третий подход — интеллектуальный анализ с применением обработки естественного языка и современных моделей машинного обучения. Данный подход предполагает использование глубоких нейронных сетей для семантического разбора текста, извлечения сущностей и автоматической многофакторной классификации. Преимущества:

- практически мгновенная обработка входящего потока информации в режиме реального времени;
- глобальный охват источников на множестве языков без необходимости пропорционального расширения штата;
- возможность автоматического геокодирования и динамической оценки уровня серьезности каждого инцидента.

Недостатки:

- высокая инженерная сложность разработки, обучения и интеграции нейросетевых моделей;
- значительная требовательность к вычислительным ресурсам, особенно при использовании больших языковых моделей;
- необходимость внедрения механизмов постоянного дообучения моделей на свежих данных для сохранения точности.

В результате проведенного анализа подходов к решению задачи, наиболее предпочтительным к реализации признан третий подход, дополненный гибридными механизмами автоматической проверки достоверности, что позволяет эффективно объединить скорость алгоритмов с качеством экспертной аналитики. Важно отметить, что применение исключительно нейросетевых моделей сопряжено с проблемой так называемого «дрейфа концептов» (concept drift). Язык описания угроз в мировых СМИ постоянно эволюционирует: регулярно появляются новые термины для обозначения кибератак, специфические названия новых вирусных штаммов или аббревиатуры недавно сформированных локальных вооруженных группировок, на которых базовая модель физически не могла быть обучена.

Для компенсации этого эффекта гибридный подход подразумевает глубокую интеграцию классических детерминированных правил и математических эвристик поверх вероятностного нейросетевого вывода. Например, если трансформерная модель сомневается в итоговой классификации события из-за нестандартной формулировки журналиста, но в тексте с помощью регулярных выражений обнаружены жестко заданные маркеры экстремальной опасности (такие как уровень магнитуды землетрясения в баллах или радиус эвакуации в километрах), система принудительно повышает приоритет инцидента. Сравнение различных методологий обработки информации показало, что традиционные методы изолированно абсолютно не справляются с современными объемами данных и требованиями к скорости оповещения. Интеллектуальный подход на базе обработки естественного языка позволяет решить проблему масштабируемости и многоязычности, обеспечивая при этом высокий уровень точности за счет использования современных трансформерных архитектур. Таким образом, внедрение гибридных технологий машинного обучения является безальтернативным путем для создания гибкой, надежной и конкурентоспособной системы глобального мониторинга рисков.

Таблица 1.1 – Сравнительный анализ систем мониторинга рисков в путешествиях

Критерий сравнения	Int. SOS	Safeture	Crisis24
Основной источник данных	Эксперты	Агрегаторы	Гибридный
Скорость и метод анализа данных	Медленный (ручной труд)	Отсутствует (только сбор)	Средний (частично ручной)
Скорость оповещения	Средняя	Высокая	Высокая
Поддержка индивидуальных путешественников (B2C)	Нет	Нет	Ограничена
Наличие NLP-конвейера (глубокий анализ текста)	Нет (ручной анализ)	Нет	Частично
Автоматическое геокодирование	Нет	Да	Да
Интеграция с картами	Да	Да	Да
Локализация данных и интерфейса	Частичная	Частичная	Частичная
Стоимость решения	Очень высокая	Высокая	Высокая

изучение коммерческих аналогов позволило выявить общую тенденцию к высокой стоимости и жесткой ориентации на корпоративный сектор. Эти системы не поддерживают услуги для индивидуальных путешественников, оставляя сегмент частных лиц незащищенным. Кроме того, большинство платформ либо полагаются на медленный ручной труд аналитиков (что снижает скорость анализа данных), либо выступают посредниками-агрегаторами без глубокого анализа. Разрабатываемый проект призван устранить эти недостатки за счет внедрения собственного аналитического конвейера на базе обработки естественного языка (NLP). Это обеспечит независимость от внешних провайдеров, гарантирует сверхвысокую скорость обработки данных и снизит эксплуатационные расходы.

1.5 Анализ источников данных

Для обеспечения полноценного глобального покрытия система должна агрегировать и обрабатывать информацию из множества гетерогенных и распределенных источников. Были проанализированы следующие ключевые категории:

- **проект GDELT (Global Database of Events, Language, and Tone)[2]** — крупнейший в мире открытый массив данных о событиях, обновляемый каждые 15 минут и содержащий ссылки на новости на 65 языках;

- **социальные медиа и мессенджеры (Twitter, Telegram)** — источники наиболее оперативной, но при этом характеризующейся экстремально высоким уровнем шума и дезинформации информации;

- **официальные правительственные отчеты и бюллетени (Министерства иностранных дел, МЧС)**: обладают высочайшей степенью достоверности, но зачастую публикуются с критической задержкой;

- **специализированные метеорологические и сейсмические службы (GDACS, USGS)** — источники, предоставляющие четко структурированные данные о природных катаклизмах в машиночитаемых форматах XML и JSON.

В рамках данной работы научно обосновано использование GDELT как основного информационного фундамента системы, так как этот проект обеспечивает беспрецедентную широту охвата и глубину метаданных, необходимых для дообучения нейронных сетей. Выбор GDELT в качестве базового источника обусловлен его уникальной способностью в режиме реального времени консолидировать мировые новости из десятков тысяч локальных изданий с автоматическим переводом на английский язык, что радикально упрощает разработку унифицированных моделей обработки естественного языка. Особую аналитическую ценность представляет встроенная в GDELT система кодирования событий CAMEO (Conflict and Mediation Event Observations), которая автоматически присваивает новостным сюжетам строгие числовые коды взаимодействий между акторами, а также использование шкалы Гольдштейна (Goldstein Scale) для базовой математической оценки потенциального влияния события на политическую стабильность конкретного региона.

Однако промышленная работа с массивами GDELT сопряжена с рядом серьезных инженерных вызовов. Наличие колоссального объема информационного шума, дублирующихся сообщений об одном и том же инциденте от разных информационных агентств и исторических смещений в данных требует разработки сложного многоэтапного конвейера дедупликации. Зачастую исторические ретроспективы или объемные аналитические статьи могут ошибочно маркироваться базовыми алгоритмами как текущие

инциденты. Для компенсации этих недостатков требуется внедрение дополнительных эвристических фильтров по временным меткам извлекаемых сущностей.

В свою очередь, прямая обработка неструктурированных социальных медиа и профильных мессенджеров требует решения сложнейших проблем, связанных со строгими ограничениями программных интерфейсов (API Rate Limits), постоянной борьбой с бот-трафиком и целенаправленными информационными вбросами (дезинформацией). Поэтому интеграция подобных источников целесообразна только при наличии мощного программного модуля оценки достоверности (Trust Scoring) на базе графового анализа надежности источников. Интеграция же данных из строго структурированных специализированных сейсмических и метеорологических служб (например, USGS, EMSC) позволит дополнить систему точными сведениями о природных угрозах, создавая надежную многослойную информационную модель окружающего мира, где вероятностные текстовые данные органично накладываются на детерминированные физические измерения.

1.6 Анализ требований к локализации данных и интерфейса

Глобальный характер системы мониторинга рисков делает локализацию не вспомогательной функцией пользовательского интерфейса, а одним из базовых требований к качеству информирования. Одно и то же событие может быть обнаружено в статье BBC на английском языке, в сообщении местного издания на испанском, в официальном бюллетене на арабском или в региональном источнике на русском языке, при этом пользователь должен получить понятное описание угрозы на выбранном языке без необходимости самостоятельно интерпретировать оригинальный текст. В контексте безопасности путешественников языковой барьер напрямую влияет на скорость принятия решений, так как неточный или запоздалый перевод может привести к неправильной оценке зоны риска, уровня опасности и рекомендуемого поведения.

Локализация системы должна охватывать два уровня. Первый уровень связан с локализацией интерфейса, то есть переводом меню, фильтров, категорий угроз, статусов маршрута, подписей на карте и шаблонов уведомлений. Второй уровень связан с локализацией данных, то есть автоматическим определением языка исходного материала, сохранением исходной версии события и генерацией переводов названий, описаний и кратких рекомендаций для поддерживаемых языков. Такой подход позволяет одновременно сохранять трассируемость к оригинальному источнику и предоставлять пользователю адаптированное содержание.

Определение языка источника информации. На первом этапе

обработки необходимо установить язык текста, из которого было извлечено событие. Для этого могут применяться алгоритмы N-граммной категоризации текста, основанные на сравнении частотных профилей символов или словоформ с заранее подготовленными профилями языков. Данный подход хорошо подходит для коротких новостных фрагментов, не требует больших вычислительных ресурсов и может быть реализован с помощью библиотек семейства Compact Language Detector, Lingua или аналогичных решений[3]. Его преимуществом является высокая скорость и предсказуемость, однако качество снижается для смешанных текстов, заголовков с большим количеством имен собственных, транслитерации и сообщений, в которых исходный язык частично заменен цитатами.

Альтернативой являются методы машинного обучения для определения языка, включая fastText lid.176, модели на основе символьных сверточных нейронных сетей и компактные трансформерные классификаторы[4]. FastText представляет особый интерес для данной системы, так как он распространяется как самодостаточная модель, поддерживает большое количество языков, быстро работает на центральном процессоре и может быть развернут внутри инфраструктурного контура без обращения к внешним программным интерфейсам. Более тяжелые трансформерные модели потенциально точнее на сложных случаях, но требуют больших ресурсов и хуже подходят для высокочастотного потока GDELT, где определение языка должно выполняться до основных этапов анализа.

Облачные решения, такие как Google Cloud Translation Language Detection, AWS Comprehend или Azure AI Language, обладают высоким качеством и простым программным интерфейсом, но создают зависимость от внешнего провайдера, увеличивают операционные затраты и требуют передачи фрагментов новостных материалов за пределы системы. Для дипломного проекта это является существенным ограничением, поскольку в системе могут обрабатываться маршруты, предпочтения пользователей и корреляции между событиями и зонами присутствия путешественников. Даже если сами новости являются открытыми, связанный с ними контекст обработки может иметь конфиденциальный характер.

Сравнение подходов к определению языка источника представлено в таблице 1.2, в которой сопоставлены наиболее значимые показатели: точность или F1-мера на задаче определения языка, количество поддерживаемых языков, вычислительная стоимость и возможность локального развертывания.

Автоматический перевод событий, описаний и заголовков. Задача автоматического перевода в системе отличается от обычного машинного перевода тем, что объектом обработки являются не только полные статьи, но и сжатые аналитические представления: заголовки событий, короткие описания, темы электронных писем, рекомендации и подписи

английский-испанский, когда требуется высокая скорость и предсказуемая стоимость инференса.

Облачные переводчики, такие как Google Cloud Translation, DeepL, Microsoft Translator и Amazon Translate, обеспечивают высокое качество на популярных языковых парах, быстро подключаются и снимают с команды часть инфраструктурной нагрузки. Их недостатки проявляются в долгосрочной эксплуатации: стоимость растет пропорционально объему переводимых текстов, доступность зависит от внешнего соглашения об уровне обслуживания, а политика обработки данных ограничивает контроль над тем, где и как сохраняются фрагменты исходных материалов. Для системы безопасности путешественников это особенно важно, так как перевод может выполняться для событий, связанных с конкретными маршрутами и настройками уведомлений пользователя.

Сравнение подходов к машинному переводу текстов событий представлено в таблице 1.3. Для машинного перевода наиболее часто используются метрики BLEU, chrF и COMET, а для промышленного внедрения дополнительно важны задержка инференса, стоимость обработки и контролируемость хранения данных.

Реферирование текстов из источников. Помимо перевода, система должна решать задачу реферирования исходных материалов, так как пользователю в условиях поездки невозможно читать полную новостную статью, официальный бюллетень или длинный аналитический текст. Реферирование в данном контексте отличается от обычного сокращения текста: итоговая сводка должна сохранить тип угрозы, место, временной контекст, потенциальное влияние на путешественника и практический смысл события. Если модель удалит из сводки упоминание конкретного города, района аэропорта, даты начала комендантского часа или направления эвакуации, то уведомление станет формально кратким, но практически бесполезным.

Существуют два основных подхода к автоматическому реферированию. Экстрактивное реферирование выбирает из исходного текста наиболее значимые предложения без их переформулирования. Классические методы TextRank, LexRank и LSA просты, устойчивы и хорошо объяснимы, однако они плохо адаптируют стиль текста под уведомление и часто сохраняют лишние журналистские обороты, цитаты или политический контекст, не имеющий прямого отношения к безопасности путешественника[11–13]. Более современные экстрактивные модели на основе BERT и Sentence-BERT лучше оценивают семантическую значимость предложений, но все равно ограничены тем, что не создают нового компактного описания события.

Абстрактное реферирование генерирует новую краткую формулировку, опираясь на смысл исходного текста. Для этой задачи применяются модели BART, PEGASUS, T5, mT5, mBART, LongT5, а также современные инструкционные большие языковые модели, такие как Qwen2.5, Mistral,

реферирования становится менее предсказуемым. В системе мониторинга рисков это недопустимо, так как редкие языки часто встречаются именно в локальных источниках, которые первыми сообщают о региональных инцидентах.

Поэтому рациональным является двухэтапный подход: сначала определить язык источника, затем при необходимости выполнить машинный перевод на один из опорных языков, для которого реферирующая модель показывает стабильное качество, и только после этого сформировать сводку. Такой порядок увеличивает вычислительные затраты, но делает поведение системы более управляемым. Перевод может выполняться через локально развернутые NLLB-200 или MarianMT, после чего локальная большая языковая модель Qwen2.5 или Mistral генерирует краткую сводку в заданном формате. Для языков, которые модель реферирования поддерживает напрямую, перевод можно пропускать, но этот список должен быть ограничен и подтвержден тестированием на реальных новостных материалах.

Облачные модели реферирования, включая облачные большие языковые модели и специализированные программные интерфейсы анализа текста, обладают высокой гибкостью и позволяют быстро получить качественные сводки без самостоятельного обслуживания графической инфраструктуры. Однако они требуют передачи полного текста статьи и контекста события внешнему провайдеру, а иногда и дополнительных метаданных, связанных с географией угрозы или пользовательскими настройками. В рамках системы обеспечения безопасности путешественников такая зависимость создает риск раскрытия операционного контекста, усложняет соблюдение требований конфиденциальности и делает стоимость обработки трудно прогнозируемой при резком росте потока событий.

Сравнение подходов к реферированию текстов представлено в таблице 1.4. Для данной задачи наиболее распространены метрики ROUGE-1, ROUGE-2, ROUGE-L и BERTScore, однако в системе безопасности дополнительно необходимо учитывать фактическую согласованность, сохранение именованных сущностей, длину контекста и возможность локального развертывания.

По результатам анализа таблиц 1.2, 1.3 и 1.4 для проекта целесообразно выбрать гибридный стек с локальным развертыванием: fastText `lid.176` или Lingua для первичного определения языка, облегченную версию NLLB-200 для универсального машинного перевода, MarianMT или OPUS-MT для наиболее частых языковых пар и локальную большую языковую модель Qwen2.5 или Mistral для реферирования и краткой стилистической нормализации уведомлений. Такой выбор обеспечивает конфиденциальность данных, снижает зависимость от коммерческих программных интерфейсов и позволяет сохранять полный контроль над журналированием, версионированием моделей и повторной обработкой событий при изменении требований

отдельные компоненты системы.

Микросервисная архитектура с синхронным взаимодействием через REST-интерфейсы также признана неоптимальной для данной задачи. Время обработки одного новостного текста нейронными сетями может значительно варьироваться, что при синхронных вызовах неизбежно приведет к возникновению каскадных блокировок и деградации производительности.

Выбранный итоговый подход — событийно-ориентированная микросервисная архитектура (Event-Driven Architecture) с применением промышленного брокера сообщений. Данный подход позволяет эффективно решать следующие базовые задачи:

- полная техническая и логическая изоляция процессов непрерывного сбора, ресурсоемкого интеллектуального анализа и массовой асинхронной рассылки уведомлений пользователям;
- гарантированная доставка (at-least-once delivery) и обработка всех накопленных данных даже при возникновении длительных временных сбоев или перезапусков в работе отдельных аналитических сервисов;
- возможность беспрепятственного горячего внедрения новых аналитических, геоинформационных и прогностических модулей в существующий конвейер без остановки обработки основного потока.

Для успешной реализации такой архитектуры в контексте высоконагруженной системы мониторинга необходимо строгое следование ряду специфических паттернов проектирования. В первую очередь, это паттерн обеспечения идемпотентности обработки сообщений. Поскольку в распределенных системах возможна доставка одного и того же события несколько раз из-за сетевых задержек, микросервисы анализа и оповещения должны уметь распознавать дубликаты инцидентов по уникальным идентификаторам и вычисляемым хэш-суммам контента, чтобы не перегружать конечного пользователя повторяющимися уведомлениями об одной и той же угрозе. Во-вторых, критически важно применение механизмов контроля обратного давления (Backpressure), которые на уровне брокера сообщений защищают реляционную базу данных и потребителей от переполнения оперативной памяти.

Оценка эффективности выбранной архитектурной модели.

Событийно-ориентированный подход объективно является наиболее адаптивным и отказоустойчивым для систем, работающих с крайне непредсказуемыми объемами входящих данных. Использование надежных очередей сообщений позволяет сглаживать пиковые нагрузки, неизбежно возникающие в моменты крупных мировых катаклизмов (масштабные стихийные бедствия, государственные перевороты, крупные террористические акты), когда количество публикуемых в мире профильных новостей возрастает на несколько порядков буквально за считанные минуты. В традиционной монолитной системе с синхронным взаимодействием подобный информаци-

онный всплеск неминуемо привел бы к быстрому исчерпанию пула потоков соединения с базой данных и полному отказу в обслуживании всех активных пользователей (DDoS-эффект). Выбранная же микросервисная архитектура обеспечивает высочайшую живучесть платформы: избыточные сообщения безопасно буферизуются в очередях на диске, а оркестратор контейнеров автоматически масштабирует только те компоненты, которые испытывают наибольшую нагрузку (например, инстансы с графическими ускорителями для инференса больших языковых моделей), никак не затрагивая при этом стабильность работы клиентского картографического интерфейса или критически важного модуля авторизации пользователей.

1.8 Выбор моделей машинного обучения

Для реализации качественного интеллектуального анализа неструктурированного текста детально рассмотрены и выбраны следующие технологические решения:

- извлечение именованных сущностей: для поиска упоминаний локаций и организаций выбраны архитектуры на основе трансформеров, демонстрирующие высокую точность на зашумленных текстах[20];

- геопарсинг и разрешение топонимических неоднозначностей: задача сопоставления текстового названия города с его точными географическими координатами с учетом широкого контекста статьи[21];

- классификация типов угроз: использование метода классификации без предварительного обучения на целевых классах на базе модели DeBERTa-v3 позволяет системе успешно распознавать новые типы угроз без необходимости проведения полного переобучения[22];

- интеллектуальное реферирование: для автоматической генерации кратких и понятных уведомлений применяются компактные языковые модели, работающие после прямой обработки поддерживаемого языка или после машинного перевода на опорный язык;

- определение языка и перевод: для локализации событий применяются локально развернутые модели fastText lid.176, NLLB-200 и MarianMT, позволяющие не передавать тексты и пользовательский контекст внешним провайдерам.

Выбранные методы интеллектуального анализа событий представлены в таблице 1.5. В качестве критериев использованы типовые метрики соответствующих задач: F1-мера для извлечения сущностей и классификации, точность и макро-F1 для определения языка, BLEU, chrF и COMET для перевода, ROUGE и BERTScore для реферирования, а также эксплуатационные показатели задержки, вычислительной стоимости и возможности локального развертывания.

Таблица 1.5 показывает, что выбранный стек моделей машинного

варительного обучения на целевых классах на базе DeBERTa-v3 выбран именно потому, что позволяет адаптировать систему к новым типам угроз простым изменением текстовых меток без полного переобучения.

Для локализации и реферирования решающими становятся другие критерии: языковое покрытие, задержка инференса, стоимость эксплуатации и конфиденциальность. Таблицы 1.2, 1.3 и 1.4 показывают, что облачные программные интерфейсы удобны на этапе прототипирования, но не соответствуют требованию полного контроля над текстами источников и пользовательским контекстом. Поэтому в качестве базового решения выбран локальный конвейер: fastText lid.176 определяет язык, NLLB-200 или MarianMT выполняет перевод на опорный язык, а Qwen2.5 или Mistral формирует краткую сводку и локализованные уведомления через локальную среду инференса Ollama[23]. Такая схема не является минимальной по числу компонентов, но она снижает риск раскрытия данных, позволяет повторно обрабатывать события при обновлении моделей и дает возможность измерять качество каждого этапа отдельными метриками.

1.9 Анализ технологического стека

Для реализации системы выбран современный и надежный технологический стек, обеспечивающий высокую производительность и удобство разработки:

- Java и фреймворк Spring Boot — для разработки высоконагруженного ядра системы, управления базой пользователей и реализации бизнес-логики[24–26];

- Python и библиотеки FastAPI, PyTorch — для построения быстрых и эффективных микросервисов машинного обучения и обработки естественного языка[27–30];

- PostgreSQL с расширением PostGIS — в качестве основной СУБД, позволяющей выполнять сложные пространственные запросы на пересечение маршрутов и зон риска непосредственно на уровне базы данных[31; 32];

- RabbitMQ — для организации надежной асинхронной связи и маршрутизации сообщений между всеми компонентами распределенной системы[33; 34];

- React и библиотека MapLibre GL JS — для создания отзывчивого клиентского интерфейса с высокопроизводительной отрисовкой векторных карт и инцидентов[35–38].

Сформированный технологический стек объединяет в себе лучшие практики корпоративной разработки на Java и передовые достижения в области искусственного интеллекта на Python. Выбор PostGIS как географического движка является ключевым проектным решением, позволяющим эффективно обрабатывать миллионы геопространственных объектов. Данная ком-

бинация технологий обеспечивает необходимый запас производительности для глобального масштабирования системы и гарантирует долгосрочную поддержку проекта.

1.10 Вывод по анализу подходов к разработке

В ходе аналитического этапа проектирования проведена комплексная работа по изучению предметной области управления рисками и существующих технологических решений. Анализ показал, что создание доступной интеллектуальной системы мониторинга на базе данных из открытых источников и современных моделей обработки естественного языка является актуальной и востребованной задачей. Обоснованный выбор событийно-ориентированной архитектуры и гибридного технологического стека закладывает надежный фундамент для реализации функциональных требований к системе, обеспечивая требуемую скорость оповещения и точность классификации угроз. Дополнительный анализ локализации показал, что поддержка нескольких языков должна быть реализована на уровне данных, интерфейса и каналов оповещения, а приоритет локально развернутых моделей оправдан требованиями конфиденциальности, контролируемости и независимости от внешних облачных программных интерфейсов.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ МОНИТОРИНГА РИСКОВ И ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПУТЕШЕСТВЕННИКОВ

В данном разделе приведено описание функциональных требований, архитектурное проектирование компонентов системы и детальное описание структуры данных и алгоритмов взаимодействия. Проектирование выполнено с учетом выявленных в ходе анализа требований к масштабируемости, отказоустойчивости и скорости обработки данных.

2.1 Функциональные требования и сценарии использования

Система мониторинга рисков спроектирована как многопользовательская платформа, обеспечивающая непрерывный цикл защиты путешественника. Основные функциональные возможности были разделены на несколько уровней взаимодействия.

Ключевые функциональные требования включают в себя:

- **управление учетными данными** — безопасная регистрация, авторизация через современные протоколы и гибкая настройка персонального профиля безопасности;
- **географическая визуализация** — интерактивное отображение глобальной карты инцидентов с возможностью фильтрации по времени, категориям (например, только медицинские или только транспортные риски) и уровню значимости;
- **динамический анализ маршрутов** — возможность загрузки планируемого пути следования (в формате GPX или через построение на карте) для автоматической проверки на наличие пересечений с активными зонами угроз;
- **проактивная система оповещений** — генерация и доставка мгновенных уведомлений в реальном времени при обнаружении новых инцидентов в радиусе ответственности пользователя;
- **локализация интерфейса и данных** — хранение предпочтительного языка пользователя, отображение интерфейса на выбранном языке и выдача локализованных названий, описаний и почтовых уведомлений;
- **автоматическое реферирование источников** — формирование краткой аналитической сводки события на основе текста исходной статьи или ее машинного перевода на опорный язык;
- **мониторинг инфраструктуры** — инструменты для администратора, позволяющие отслеживать работоспособность микросервисов, нагруз-

Использование Docker позволяет изолировать Python-окружения с тяжелыми библиотеками машинного обучения от Java-среды серверной части, обеспечивая стабильность и предсказуемость поведения системы в различных инфраструктурных окружениях.

2.2.1 Модуль сбора данных

Данный микросервис реализован на языке Python и оптимизирован для выполнения длительных сетевых операций ввода-вывода. Его основными функциями являются:

- **планирование обновлений** — согласно регламенту GDELT 2.0, новые архивы публикуются каждые 15 минут, поэтому модуль использует асинхронный цикл для загрузки и распаковки данных;
- **многоэтапная фильтрация** — для снижения нагрузки на вычислительные ресурсы анализатора модуль сбора данных проводит первичную грубую очистку данных, отсекая события с низким индексом достоверности или находящиеся вне зон интереса системы;
- **обеспечение живучести** — все скачанные сырые данные сохраняются в промежуточную базу данных, что гарантирует возможность повторного анализа при обновлении версий нейросетевых моделей;
- **сигнализация событий** — после успешной подготовки порции данных модуль публикует идентификатор события в RabbitMQ, инициируя дальнейшую цепочку обработки.

2.2.2 Модуль интеллектуального анализа событий

Это наиболее сложный компонент системы, реализующий современный конвейер обработки естественного языка. В отличие от классических систем, использующих жесткие правила, модуль интеллектуального анализа опирается на семантическое понимание текста. Алгоритм работы модуля детально представлен на рисунке 2.4.

Архитектурные и ИИ-решения, примененные в модуле:

- **классификация без предварительного обучения на целевых классах** — использование предобученной модели DeBERTa-v3 позволяет классифицировать события по произвольному набору меток без переобучения, это критически важно для адаптации системы к новым видам гибридных угроз;
- **контекстный геопарсинг** — интеграция библиотеки Mordecai позволяет не просто находить названия городов, но и разрешать топонимические конфликты (например, отличать Лондон в Великобритании от Лондона в Канаде) на основе анализа окружающего текста;
- **локализационный контур** — определение языка исходной статьи

модель стабильно выполняет краткое реферирование, текст передается в модель напрямую. Если язык не входит в поддерживаемый набор, модуль сначала использует локальную модель перевода, например NLLB-200 или MarianMT, переводит материал на английский или русский язык, а затем формирует краткую аналитическую сводку. Такой порядок снижает риск некорректной генерации на редких языках и позволяет контролировать качество результата через ограниченный набор проверенных языковых направлений.

2.2.3 Серверная часть приложения

Микросервис на Java 21, управляющий бизнес-логикой и интеграцией с конечными пользователями. Его основными функциями являются:

- **геопространственная обработка** — использование библиотеки Hibernate Spatial в связке с PostGIS позволяет выполнять операции пространственного пересечения и поиска по расстоянию с высокой скоростью на миллионах записей;
- **алгоритм асинхронного геофенсинга** — запуск фоновой задачи по поиску пересечений зоны риска с маршрутами активных пользователей при поступлении нового события, что позволяет не блокировать основной поток обработки;
- **безопасность доступа** — реализация авторизации без хранения состояния на основе JWT-токенов с интеграцией через Spring Security и защитой от перебора паролей.

2.3 Взаимодействие компонентов системы

Для обеспечения согласованности данных в распределенной среде используется паттерн асинхронного взаимодействия. На рисунке 2.5 представлена диаграмма последовательности обработки события, иллюстрирующая полный жизненный цикл данных — от публикации новости до уведомления путешественника.

Анализ временных диаграмм показывает, что узким местом является этап работы больших языковых моделей, поэтому для обеспечения масштабируемости предусмотрена возможность горизонтального увеличения количества экземпляров модуля интеллектуального анализа.

2.4 Проектирование конвейера реферирования и локализации

Конвейер подготовки текстового представления события проектируется как последовательность контролируемых этапов, каждый из которых сохраняет промежуточный результат и может быть повторно выполнен при изменении модели или правил обработки. На вход поступает исходный текст статьи, полученный из внешнего источника или из промежуточного хранилища. Далее выполняется определение языка, проверка поддерживаемости языка моделью реферирования, машинный перевод при необходимости, генерация краткой сводки, локализация заголовка и описания, а затем сохранение результата в основную базу данных.

Ключевым проектным решением является отказ от попытки реферировать все языки непосредственно одной универсальной моделью. Несмотря на наличие мультязычных больших языковых моделей, качество их работы на редких языках и смешанных текстах остается нестабильным. Поэтому в системе вводится понятие опорного языка: если исходный материал написан на английском или русском языке, он может быть передан в локальную модель реферирования напрямую, а для остальных языков сначала выполняется машинный перевод локально развернутой моделью. Такой подход делает качество сводок более предсказуемым и упрощает тестирование, так как проверять необходимо ограниченное число языковых направлений.

Результат реферирования хранится отдельно от полного описания события. Полное описание используется на странице события и в аналитическом интерфейсе, а краткая сводка применяется в списке событий, теме электронного письма, мгновенном уведомлении и компактной карточке на карте. Для этого в сущности **Event** предусмотрено поле **summary_i18n**, организованное по тому же принципу, что **title_i18n** и **description_i18n**. Если нужная локаль отсутствует, серверная часть использует резервную версию на опорном языке, но сам факт отсутствия перевода фиксируется в журнале обработки для последующей повторной генерации.

Для предотвращения искажения смысла в сводке проектируется дополнительная проверка ключевых сущностей. После генерации краткого текста система сопоставляет найденные в нем географические названия, даты, числовые значения и тип угрозы с результатами предыдущих этапов анализа. Если сводка не содержит основной локации или противоречит классификации события, результат помечается как требующий повторной

обработки с более строгим промптом или с использованием другой модели. Такой механизм особенно важен для уведомлений, поскольку пользователь принимает решение на основе одного-двух предложений.

С точки зрения производительности реферирование выполняется асинхронно и не блокирует прием новых сообщений из очереди. Для критических событий сначала может быть сохранена минимальная карточка с категорией, геометрией и ссылкой на источник, а затем, после завершения перевода и реферирования, запись обновляется локализованными текстовыми полями. Это позволяет не задерживать геопространственную обработку и при этом постепенно повышать качество пользовательского представления события.

2.5 Проектирование структуры базы данных

База данных является фундаментом системы и спроектирована с учетом требований к хранению сложных географических объектов и максимальной производительности ГИС-запросов. Диаграмма классов представлена на рисунке 2.6.

На диаграмме изображены:

- сущность **Event** представляет инцидент, хранящий геометрию зоны риска, локализованные текстовые поля и рассчитанный уровень угрозы;
- сущность **User** представляет профиль пользователя, хранящий электронную почту, предпочтительную локаль и настройки приватности, включая флаги разрешения на отслеживание местоположения;
- сущность **UserLocation** представляет текущее положение пользователя, необходимое для оперативной проверки попадания в активную зону риска;
- сущность **Route** представляет маршрут пользователя, хранящий геометрию пути и временные метки прохождения ключевых точек;
- сущность **Mention** представляет упоминание события во внешнем источнике, содержащее ссылку, язык текста и дату публикации материала;
- сущность **Source** представляет справочник источников данных, позволяющий связывать событие с конкретным новостным доменом или агрегатором;
- сущность **EventType** представляет справочник типов событий, определяющий базовую категорию угрозы и радиус зоны влияния по умолчанию.

Для хранения локализованного содержания событий выбран тип JSONB в PostgreSQL. В отличие от отдельной таблицы переводов, JSONB позволяет хранить компактный объект, в котором ключами выступают языковые коды, а значениями являются локализованные строки, непосредственно в записи события, что уменьшает количество соединений при чтении карты и страниц инцидентов. При этом PostgreSQL сохраняет возможность индексирования ключей JSONB, проверки наличия конкретной локали и частичного обновления переводов без пересоздания всей строки.

В сущности **Event** предусмотрены поля **title_i18n**, **description_i18n** и **summary_i18n**, где ключами выступают ISO-коды языков, а значениями являются локализованные версии названия, описания и краткой сводки. Дополнительно хранится **source_language**, фиксирующий язык исходной статьи, чтобы аналитический модуль мог оценивать качество перевода, а пользователь или администратор могли вернуться к первичному тексту при спорных случаях. Для пользовательского профиля в сущности **User** вводится поле **preferred_locale**, которое используется серверной частью при выборе языка ответа программного интерфейса и при генерации шаблонов почтовых уведомлений.

Выбор JSONB является компромиссом между гибкостью и нормализацией. На раннем этапе проекта список поддерживаемых языков может расширяться, поэтому жесткая схема с колонками **title_ru**, **title_en** и аналогичными полями быстро приведет к миграциям при добавлении новых локалей. Отдельная таблица переводов удобна для очень больших словарей,

но в данном случае каждое событие содержит ограниченное количество текстовых фрагментов, которые почти всегда читаются вместе с основной записью, поэтому JSONB снижает сложность запросов и хорошо соответствует модели выдачи данных для карты, страницы события и уведомлений.

2.6 Проектирование программных интерфейсов

Для взаимодействия клиентской и серверной частей используется программный интерфейс в стиле REST, соответствующий спецификации OpenAPI 3.0, что обеспечивает простоту интеграции и возможность автоматической генерации клиентских библиотек.

Основные эндпоинты системы спроектированы по принципу минимальной достаточности данных:

- **GET /api/v1/events/map** — выдает массив событий в формате GeoJSON, оптимизированном для отрисовки на мобильных устройствах;
- **GET /api/v1/events/{id}** — возвращает детальную карточку события с выбором локализованных полей по параметру **locale** или по языку профиля пользователя;
- **POST /api/v1/user/location** — обеспечивает высокочастотное обновление координат пользователя с сохранением истории для последующего анализа безопасности;
- **POST /api/v1/trips/analyze** — сложный запрос, принимающий геометрию маршрута и возвращающий детализированный отчет о потенциальных рисках на пути следования.

Для обеспечения высокой надежности взаимодействия между сервисами внутри контура используется протокол gRPC, который позволяет передавать бинарные данные с минимальными накладными расходами на сериализацию, что особенно важно при передаче больших текстов статей.

2.7 Выводы по проектированию

В процессе проектирования была разработана современная архитектура интеллектуальной системы, объединяющая микросервисный подход и передовые технологии искусственного интеллекта. Использование событийно-ориентированной модели в сочетании с Docker-контейнеризацией обеспечивает необходимый уровень масштабируемости и гибкости. Проектирование базы данных на основе PostGIS и внедрение асинхронного геофенсинга позволяет системе обрабатывать глобальные потоки данных в режиме реального времени. Сформированные функциональные требования и спроектированные программные интерфейсы закладывают основу для успешной реализации программного комплекса, способного обеспечить новый уровень безопасности для путешественников.

3 РЕАЛИЗАЦИЯ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ МОНИТОРИНГА РИСКОВ И ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПУТЕШЕСТВЕННИКОВ

В данном разделе подробно описывается процесс программной реализации спроектированной системы, приводятся примеры исходного кода ключевых модулей на языках Python и Java, а также демонстрируются результаты работы пользовательского интерфейса. Разработка велась с применением современных практик непрерывной интеграции, контейнеризации и автоматизированного тестирования, что позволило обеспечить высокую надежность и расширяемость всех компонентов интеллектуальной платформы.

Архитектура разработанного программного комплекса опирается на микросервисный подход и событийно-ориентированное взаимодействие. Выбор данной парадигмы продиктован необходимостью независимого масштабирования вычислительно сложных узлов (таких как анализ текстов нейронными сетями) и узлов, отвечающих за ввод-вывод (сбор данных, отдача картографической информации клиентам). В качестве брокера сообщений, связывающего компоненты системы, выбран RabbitMQ, что обеспечило гарантированную доставку данных между асинхронными сервисами даже в условиях пиковых нагрузок.

3.1 Разработка модуля сбора и первичной обработки данных

Модуль Fetcher является входной точкой системы и реализован на языке Python 3.12 с использованием асинхронной парадигмы программирования. Основная сложность реализации данного модуля заключалась в необходимости обработки огромных массивов данных проекта GDELT (Global Database of Events, Language, and Tone), которые публикуются в виде сжатых CSV-файлов каждые 15 минут и могут содержать информацию о тысячах событий одновременно.

Каждое обновление GDELT состоит из трех основных файлов: массива событий (Events), упоминаний в СМИ (Mentions) и графа глобальных знаний (Global Knowledge Graph, GKG). Общий объем распакованных данных за одни сутки может превышать несколько гигабайт, что делает невозможным их обработку традиционными синхронными методами без существенных задержек.

Для обеспечения высокой производительности и минимизации задержек разработан асинхронный клиент на базе библиотеки **httpx**, позволяю-

щий параллельно загружать архивы событий и упоминаний. Использование **httpx** вместо стандартной библиотеки **requests** позволило освободить глобальную блокировку интерпретатора (GIL) на время ожидания ответа от серверов GDELT, за счет чего один процесс может обрабатывать десятки сетевых соединений одновременно.

После загрузки и распаковки в оперативной памяти данные передаются в конвейер обработки, реализованный с применением библиотеки **pandas**. Данная библиотека, написанная на языке C, обеспечивает векторизованную фильтрацию и очистку данных на низком уровне. В частности, на этапе первичной фильтрации отбрасываются события, индекс Гольдштейна которых не удовлетворяет заданным порогам опасности (например, события с положительным индексом, означающие мирные инициативы, исключаются из обработки как не представляющие угрозы для путешественников).

На рисунке 3.1 представлен фрагмент кода асинхронного сервиса загрузки данных GDELT.

Отдельное внимание уделено механизмам отказоустойчивости. При взаимодействии со сторонними API часто возникают сетевые аномалии: обрывы соединений, таймауты или временная недоступность файлов. Для решения этой проблемы в модуле сбора данных реализован паттерн Exponential Backoff (экспоненциальная задержка при повторных попытках). При возникновении ошибки скачивания скрипт повторяет запрос через 2, 4, 8 и 16 секунд, прежде чем пометить файл как недоступный и отправить соответствующее уведомление в систему мониторинга.

Ключевые особенности реализации модуля сбора данных:

- применение асинхронных контекстных менеджеров для эффективного управления сетевыми соединениями и ресурсами операционной системы;
- использование векторизованных операций библиотеки **pandas** для быстрой фильтрации записей по пороговым значениям;
- реализация отказоустойчивой очереди сообщений на базе RabbitMQ (использование механизмов Publisher Confirms и dead-letter exchanges), гарантирующей доставку идентификаторов событий в аналитический модуль;
- внедрение механизмов логирования и мониторинга состояния сетевых запросов;
- использование временных хранилищ (in-memory buffers) для минимизации нагрузки на дисковую подсистему при обработке архивов.

Данный подход позволил сократить время от момента публикации данных в GDELT до начала их анализа нейронными сетями до 5–10 секунд, что является критически важным для систем экстренного оповещения и мониторинга рисков в реальном времени.

```

1 import asyncio
2 import httpx
3 import pandas as pd
4 from aio_pika import connect, Message
5
6
7 class GdeltFetcher:
8     BASE_URL = "http://data.gdeltproject.org/gdeltv2/"
9     GOLDSTEIN_THRESHOLD = -5.0
10
11     def __init__(self, rabbit_url: str, http_timeout: float = 30.0):
12         self.rabbit_url = rabbit_url
13         self.client = httpx.AsyncClient(timeout=http_timeout)
14
15     async def fetch_latest_export(self, file_name: str) -> bool:
16         url = f"{self.BASE_URL}{file_name}"
17         response = await self.client.get(url)
18         if response.status_code != 200:
19             return False
20         with open(f"/tmp/{file_name}", "wb") as fp:
21             fp.write(response.content)
22         return True
23
24     async def process_csv(self, path: str) -> list[str]:
25         df = pd.read_csv(path, sep="\t", header=None, on_bad_lines="skip")
26         critical_events = df[df[30] < self.GOLDSTEIN_THRESHOLD]
27         return critical_events[0].astype(str).tolist()
28
29     async def notify_analyzer(self, event_ids: list[str]) -> None:
30         connection = await connect(self.rabbit_url)
31         async with connection:
32             channel = await connection.channel()
33             for eid in event_ids:
34                 await channel.default_exchange.publish(
35                     Message(eid.encode()),
36                     routing_key="events_queue",
37                 )
38
39     async def run(self, file_name: str) -> None:
40         if not await self.fetch_latest_export(file_name):
41             return
42         ids = await self.process_csv(f"/tmp/{file_name}")
43         if ids:
44             await self.notify_analyzer(ids)

```

Рисунок 3.1 – Фрагмент кода асинхронного сервиса загрузки данных GDELT

3.2 Реализация интеллектуального анализа событий

Модуль интеллектуального анализа представляет собой высокотехнологичный конвейер на базе фреймворка FastAPI, объединяющий классические методы обработки естественного языка и современные генеративные модели. Процесс анализа разделен на этапы извлечения контента, семантической классификации и реферирования, что позволяет гибко настраивать

параметры каждой модели в зависимости от типа входных данных и доступных вычислительных ресурсов.

Первым этапом конвейера является извлечение полного текста новостной статьи по URL-адресу, полученному из GDELT. Для этого применяется эвристический парсер на базе библиотеки **BeautifulSoup4**, который очищает HTML-документ от рекламных баннеров, скриптов, элементов навигации и извлекает исключительно текстовое содержимое статьи. Для обхода блокировок со стороны новостных порталов используются динамические пулы User-Agent и механизмы ротации прокси-серверов.

Для классификации угроз используется энкодерная модель DeBERTa-v3-large, настроенная на задачу классификации без предварительного обучения на целевых классах (Zero-Shot Classification). В отличие от классического подхода с дообучением (Fine-Tuning), Zero-Shot классификация на базе гипотез (NLI - Natural Language Inference) позволяет системе распознавать новые типы инцидентов, динамически подставляя их в промпт. Например, система может классифицировать текст по меткам «террористический акт», «стихийное бедствие», «эпидемия» или «гражданские беспорядки» без необходимости формирования обширной обучающей выборки и затратного переобучения весов нейронной сети.

Особое внимание уделено интеграции с локальной языковой моделью через программный интерфейс Ollama. В качестве базовой генеративной модели выбрана Qwen 2.5 с квантованием до 4 бит (GGUF формат). Это решение обеспечивает полную автономность системы, исключает зависимость от внешних платных провайдеров (таких как OpenAI API или Anthropic) и гарантирует конфиденциальность обрабатываемых данных. Использование квантованных моделей позволяет запускать LLM на потребительском оборудовании с объемом видеопамати от 8 ГБ, не теряя при этом в качестве генерации аналитических сводок.

На этапе реферирования LLM получает строгий системный промпт (System Prompt), предписывающий выделить из текста суть инцидента, точное местоположение, количество пострадавших (если есть) и потенциальные последствия для транспортной инфраструктуры. Модель генерирует компактную сводку (до 500 символов) и список рекомендуемых действий для путешественников.

На рисунке 3.2 представлен фрагмент кода модуля интеллектуального анализа событий.

```

1 from transformers import pipeline
2 import httpx
3
4
5 class IntelligenceAnalyzer:
6     LABELS = ["terrorism", "disaster", "protest", "medical"]
7     SUPPORTED_LOCALES = ("en", "ru")
8
9     def __init__(self, ollama_url: str, llm_model: str = "qwen2.5:7b"):
10         self.classifier = pipeline(
11             "zero-shot-classification",
12             model="MoritzLaurer/DeBERTa-v3-large-mnli-fever-anli-ling",
13         )
14         self.ollama_url = ollama_url
15         self.llm_model = llm_model
16
17     async def analyze_text(self, text: str) -> dict:
18         source_language = self.detect_language(text)
19         scores = self.classifier(text, self.LABELS, multi_label=False)
20
21         summary = await self.generate_llm_summary(text)
22         localized = await self.localize_event(summary, self.SUPPORTED_LOCALES)
23
24         return {
25             "category": scores["labels"][0],
26             "confidence": scores["scores"][0],
27             "source_language": source_language,
28             "summary": summary,
29             "description_i18n": localized,
30         }
31
32     async def generate_llm_summary(self, content: str) -> str:
33         prompt = (
34             "Summarize this news in one short sentence: "
35             f"{content[:1000]}"
36         )
37         payload = {
38             "model": self.llm_model,
39             "prompt": prompt,
40             "stream": False,
41         }
42         async with httpx.AsyncClient(timeout=60.0) as client:
43             response = await client.post(
44                 f"{self.ollama_url}/api/generate",
45                 json=payload,
46             )
47             response.raise_for_status()
48             return response.json().get("response", "")
49
50     async def localize_event(self, summary: str, locales) -> dict:
51         return {locale: await self.translate_text(summary, locale)
52                 for locale in locales}
53
54     def detect_language(self, text: str) -> str:
55         return "en"
56
57     async def translate_text(self, text: str, locale: str) -> str:
58         return text

```

Рисунок 3.2 – Фрагмент кода модуля интеллектуального анализа событий

Для поддержки локализации в аналитический конвейер был добавлен этап определения языка исходного материала. С помощью легковесной модели **fasttext** система за миллисекунды определяет язык оригинальной статьи. После формирования базовой сводки на английском языке, модуль инициирует асинхронные запросы к LLM для подготовки локализованных

вариантов названия, описания и рекомендаций для русского, испанского и других поддерживаемых языков.

Программные решения, внедренные в аналитический модуль:

- использование библиотеки **transformers** для высокоточной семантической классификации;
- интеграция с локальным сервером Ollama через REST API для выполнения ресурсоемких задач генерации на выделенных графических мощностях;
- ограничение входного контекста для большой языковой модели до 1000 токенов с целью оптимизации времени инференса и предотвращения эффекта потери внимания (lost in the middle);
- применение пула воркеров (Worker Pool) и семафоров для контроля параллельной нагрузки на GPU, чтобы избежать ошибок out-of-memory (OOM);
- сохранение готовых локализованных версий текстов в кэш-базу данных для моментальной выдачи пользователю без повторного запуска инференса.

3.3 Реализация серверной логики и геоинформационных компонентов

Ядро системы, отвечающее за бизнес-логику, хранение данных, авторизацию пользователей и выполнение пространственных запросов, реализовано на языке Java 21 с использованием фреймворка Spring Boot 3.3. В качестве основной системы управления базами данных выступает PostgreSQL, выбор которой обусловлен её высокой надежностью, поддержкой JSON-полей и, что самое главное, наличием мощнейшего геоинформационного расширения PostGIS.

Ключевым технологическим преимуществом является использование модуля Hibernate Spatial. Данный модуль интегрируется в слой ORM (Object-Relational Mapping) и позволяет оперировать геометрическими объектами (Point, LineString, Polygon) непосредственно в коде Java-приложения, используя стандартные классы библиотеки JTS (Java Topology Suite). При сохранении сущности в базу данных Hibernate Spatial автоматически транслирует JTS-объекты в формат WKB (Well-Known Binary), понятный базе данных PostGIS.

Для реализации логики геофенсинга — поиска пользователей и маршрутов, находящихся в зоне риска, — применяются пространственные функции PostGIS, такие как **ST_DWithin** (проверка нахождения объектов в пределах заданного радиуса) и **ST_Intersects** (проверка пересечения геометрий). Использование пространственных индексов типа GiST (Generalized Search Tree) позволяет выполнять такие запросы за логарифмическое время

даже на таблицах с миллионами записей инцидентов. Выполнение геометрических вычислений на стороне СУБД радикально снижает объем данных, передаваемых по сети между базой и приложением, так как в Java-код возвращается только готовый отфильтрованный результат.

Сущности базы данных описывают структуру инцидента и одновременно обеспечивают поддержку многоязычности. Вместо создания множества связанных таблиц для каждого языка, применен современный подход с использованием типа данных **JSONB**. В одном поле сущности хранится JSON-объект, ключами которого выступают коды языков (en, ru, es), а значениями — локализованные строки. При выборке данных база может извлекать конкретную локаль на лету с помощью функции `jsonb_extract_path_text`.

Поверх сущностей построен гибкий слой репозитория на базе JPA Criteria API. Метод `findEventsForMap` динамически конструирует SQL-запрос, применяя фильтры по bounding box (видимой области экрана пользователя), минимальному уровню угрозы, категориям инцидентов и времени публикации. На рисунке 3.3 представлен фрагмент кода репозитория пространственной выборки событий.

Для отдачи геоданных клиентскому приложению реализован унифицированный контроллер **GeoServerController**, который агрегирует информацию от разных сервисов-поставщиков (инциденты, сохраненные маршруты, геозоны) и формирует единый валидный GeoJSON-ответ.

В высоконагруженных сценариях частое обновление карты пользователями может создать чрезмерную нагрузку на СУБД. Для предотвращения этого в серверную часть интегрирован распределенный кэш на базе Redis. Сформированные GeoJSON-ответы кэшируются по ключам, представляющим собой хэш от параметров запроса (координаты углов карты, фильтры). Время жизни кэша (TTL) настроено на 30 секунд, что обеспечивает идеальный баланс между актуальностью данных и производительностью сервера.

3.4 Реализация системы почтовых уведомлений

Для гарантированной доставки экстренной информации об угрозах в системе реализована асинхронная рассылка по электронной почте. Этот канал рассматривается как резервный, независимый от работы веб-приложения и мобильных клиентов, а также юридически значимый способ оповещения.

Сервис уведомлений на Java использует библиотеку **Spring Boot Starter Mail** для взаимодействия с внешним SMTP-сервером. Чтобы длительное ожидание ответа от почтового сервера не блокировало основной поток выполнения бизнес-логики, методы отправки аннотированы как `@Async`, и их выполнение делегируется пулу потоков

```

1 @Repository
2 public class CriteriaEventGeofeatureRepository
3     implements EventGeofeatureRepository {
4
5     @PersistenceContext
6     private EntityManager entityManager;
7
8     public List<EventGeofeatureDto> findEventsForMap(
9         EventGeofeatureFilter filter) {
10
11         CriteriaBuilder cb = entityManager.getCriteriaBuilder();
12         CriteriaQuery<EventGeofeatureDto> query =
13             cb.createQuery(EventGeofeatureDto.class);
14         Root<EventEntity> root = query.from(EventEntity.class);
15
16         List<Predicate> predicates = new ArrayList<>();
17
18         if (filter.getMinLon() != null && filter.getMaxLon() != null) {
19             Expression<?> envelope = cb.function(
20                 "ST_MakeEnvelope", Object.class,
21                 cb.literal(filter.getMinLon()),
22                 cb.literal(filter.getMinLat()),
23                 cb.literal(filter.getMaxLon()),
24                 cb.literal(filter.getMaxLat()),
25                 cb.literal(4326));
26             predicates.add(cb.isTrue(cb.function(
27                 "ST_Intersects", Boolean.class,
28                 root.get("epicenter"), envelope)));
29         }
30
31         if (filter.getMaxSeverity() != null) {
32             predicates.add(cb.lessThanOrEqualTo(
33                 root.get("severity").get("severityValue"),
34                 filter.getMaxSeverity().getSeverityValue()));
35         }
36
37         predicates.add(cb.or(
38             cb.isNull(root.get("expiresAt")),
39             cb.greaterThan(root.get("expiresAt"),
40                 LocalDateTime.now())));
41
42         Expression<String> localizedTitle = cb.function(
43             "jsonb_extract_path_text", String.class,
44             root.get("title"), cb.literal(filter.getLang()));
45         Expression<String> geoJson = cb.function(
46             "ST_AsGeoJSON", String.class,
47             root.get("epicenter"));
48
49         query.select(cb.construct(EventGeofeatureDto.class,
50             root.get("id"),
51             localizedTitle,
52             root.get("severity").get("severityValue"),
53             geoJson));
54         query.where(predicates.toArray(new Predicate[0]));
55
56         return entityManager.createQuery(query).getResultList();
57     }
58 }

```

Рисунок 3.3 – Фрагмент кода репозитория пространственной выборки событий

ThreadPoolTaskExecutor.

Отдельным сложным этапом разработки стала верстка шаблонов уведомлений. Почтовые клиенты (особенно корпоративные, такие как Microsoft Outlook) используют устаревшие механизмы рендеринга HTML и не поддерживают современные стандарты CSS. Поэтому шаблоны сверстаны с

автоматически кэшировать запросы на клиенте, управлять состояниями загрузки и минимизировать дублирование сетевых вызовов.

Центральным элементом пользовательского интерфейса является интерактивная карта. Для её отрисовки используется библиотека **react-map-gl**, представляющая собой React-обертку над высокопроизводительным движком **MapLibre GL JS**. Выбор MapLibre обусловлен его открытой лицензией, поддержкой аппаратного ускорения через WebGL и возможностью работы с векторными тайлами, что выгодно отличает его от растровых решений типа Leaflet.

Для оптимизации рендеринга больших объемов геоданных (десятков тысяч инцидентов) на стороне клиента реализована кластеризация. При отдалении камеры близко расположенные маркеры объединяются в кластеры с отображением количества сгруппированных событий. Это предотвращает падение FPS (частоты кадров) в браузере и делает интерфейс отзывчивым. Применение концепции Data-Driven Styling позволяет прямо в конфигурации слоев карты указывать правила окрашивания маркеров в зависимости от атрибута уровня угрозы (`risk_level`) без написания дополнительных скриптов.

Перед началом работы с системой пользователь проходит процедуру аутентификации, реализованную на базе JWT-токенов (JSON Web Token). Окно входа выполнено в минималистичном стиле и поддерживает мгновенное переключение языка. После авторизации токен сохраняется в безопасном хранилище браузера (HttpOnly куки), а пользователь перенаправляется на главный экран.

На рисунке 3.5 представлено окно входа в систему, а на рисунке 3.6 — главное окно пользовательского интерфейса системы.

Для детального изучения отдельного инцидента реализована страница события (или всплывающая боковая панель). На ней отображается локализованный заголовок, агрегированный уровень угрозы, аналитическая сводка от LLM и рекомендации. Поскольку текстовое содержимое уже было сформировано на бэкенде, клиентское приложение лишь подставляет нужные поля из JSON-ответа, что делает переключение языка интерфейса абсолютно бесшовным и мгновенным. На рисунке 3.7 представлена страница события для русскоязычного пользователя.

3.6 Разработка интерфейса прокладки и анализа маршрута

Важнейшей аналитической функцией системы является модуль интерактивного планирования маршрута. Пользовательский сценарий реализован в виде пошагового мастера. На первом шаге пользователь добавляет путевые точки кликами по карте или через строку поиска (геокодирование). На вто-

4 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ И ИСПОЛЬЗОВАНИЯ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ МОНИТОРИНГА РИСКОВ И ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПУТЕШЕСТВЕННИКОВ ПО ИНДИВИДУАЛЬНОМУ ЗАКАЗУ

4.1 Характеристика разработанного по индивидуальному заказу программного средства интеллектуального мониторинга рисков и обеспечения безопасности путешественников

Целью дипломного проекта является разработка интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников по индивидуальному заказу туристической компании ООО «ТревелСейф».

Целью разработки интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников является автоматизация процесса сбора, семантического анализа и оперативного оповещения об угрозах для обеспечения принципа «должной осмотрительности» (Duty of Care) и защиты клиентов и сотрудников компании во время поездок.

Преимуществами данного программного средства являются сверх-высокая скорость обработки данных за счет событийно-ориентированной микросервисной архитектуры, использование передовых моделей машинного обучения (NLP, Zero-shot классификация, LLM) для глубокого анализа открытых источников без ручной модерации, а также интуитивно понятный интерфейс с интерактивной глобальной картой, простота использования и широкий функционал.

Данное программное средство будет использоваться в компании ООО «ТревелСейф» для непрерывного мониторинга глобальных инцидентов, оценки безопасности маршрутов и автоматического геофенсингового контроля путешественников.

Основные функции, которые выполняет программное средство:

- авторизация пользователя и системного администратора;
- регистрация и настройка профиля пользователя (установка предпочтений по типам угроз и критическим уровням опасности);
- непрерывный автоматизированный сбор данных из базы GDELT и других открытых источников;
- интеллектуальный семантический анализ текстовой информации (классификация типов угроз, геопарсинг локаций, реферирование новостей)

с помощью ИИ);

- отображение глобальной интерактивной карты рисков в режиме реального времени;
- интеллектуальный мониторинг планируемого маршрута на предмет пространственного пересечения с зонами активных инцидентов;
- применение алгоритмов геофенсинга для отслеживания местоположения пользователей;
- автоматическая рассылка push-уведомлений при вхождении пользователя в зону риска или при возникновении критического события в радиусе нахождения пользователя;
- администрирование и мониторинг состояния сервисов системы (обработка очередей сообщений, статусы загрузки данных).

Экономическая оценка целесообразности инвестиций в разработку и использование программного средства осуществляется на основе расчета и оценки следующих показателей: чистый дисконтированный доход, рентабельность инвестиций и срок окупаемости инвестиций.

4.2 Расчет затрат на разработку и цены интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников, созданной по индивидуальному заказу

Для разработки проекта компания-заказчик заключила соглашение с компанией-разработчиком на разработку программного средства. В соглашении определены различные требования к разрабатываемому программному средству и его цена. Цена программного средства определена на основе полных затрат на разработку программного средства организацией-разработчиком и включает в себя следующие статьи затрат: основная заработная плата разработчиков, дополнительная заработная плата разработчиков, отчисления на социальные нужды, прочие расходы, общая сумма затрат на разработку, плановая прибыль (включаемая в цену программного средства), отпускная цена программного средства.

4.2.1 Расчет затрат на основную заработную плату разработчикам

Для расчета затрат на разработку программного средства в первую очередь необходимо рассчитать основную заработную плату команды разработчиков. Расчет осуществляется исходя из состава и численности команды, размера месячной заработной платы каждого участника команды, а также трудоемкости работ, выполняемых при разработке программного средства отдельными исполнителями по формуле:

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта успешно спроектирована и реализована интеллектуальная система мониторинга рисков и обеспечения безопасности путешественников, объединяющая передовые методы машинного обучения, геоинформационные технологии и высоконагруженную микросервисную архитектуру.

На этапе анализа предметной области детально исследованы существующие подходы организации поездок, изучены международные стандарты управления рисками (в частности, ISO 31030). Проведен сравнительный анализ ведущих мировых информационно-аналитических платформ и коммерческих аналогов разрабатываемой системы. Были выявлены их ключевые недостатки: высокая стоимость владения, закрытость экосистем, невозможность локального контроля данных и высокая зависимость от ручного труда операторов. На основе полученных аналитических данных были выдвинуты и формализованы строгие функциональные и нефункциональные требования к разрабатываемой платформе, ориентированные на высокую скорость реакции и автономность работы.

На этапе проектирования проведен анализ профилей конечных пользователей системы, определены их основные потребности и сценарии взаимодействия с системой. Сформирован оптимальный стек технологий и обоснован выбор моделей машинного обучения. Спроектирована масштабируемая событийно-ориентированная архитектура системы на базе промышленного брокера сообщений, разработана концептуальная, логическая и физическая структура базы данных с поддержкой геопространственных индексов. Отдельное внимание было уделено интернационализации данных и интерфейса.

На этапе реализации создан полностью функционирующий отказоустойчивый комплекс, состоящий из набора независимых микросервисов. На языке Python 3.12 разработан высокопроизводительный асинхронный модуль сбора данных а также инновационный конвейер интеллектуального анализа текстов. Серверная часть бизнес-логики разработана на платформе Java 21 с применением фреймворка Spring Boot. Клиентское веб-приложение, построенное на базе библиотеки React 19 и движка MapLibre GL JS. Проведено мануальное тестирование и отладка всего функционала системы, по результатам которых система была признана полностью работоспособной.

На этапе экономического обоснования тщательно проанализированы ресурсы, необходимых для реализации и внедрения системы. Составлена смета капитальных затрат и рассчитаны ожидаемые операционные расходы на оплату труда инженерного состава. Проведенные финансовые расчеты показали, что разработка данной системы является экономически целесообразной. Срок полной окупаемости проекта составит около 2 лет.

В качестве направлений для дальнейшего совершенствования системы рассматриваются следующие стратегические шаги:

- замена ручного мастера прокладки маршрутов на автоматизированную интеграцию с программными интерфейсами (API) популярных букинг-провайдеров;
- расширение количества поддерживаемых языков интерфейса и уведомлений;
- разработка нативного мобильного приложения для платформ iOS и Android.

В настоящее время разработанная система полностью готова к этапу промышленной эксплуатации. Ведутся подготовительные работы по её внедрению в отделы по туристической и корпоративной безопасности нескольких крупных организаций.

Таким образом, можно констатировать, что все цели дипломного проектирования достигнуты в полном объеме, а поставленные практические задачи успешно решены на высоком уровне. Полученные результаты обладают значительной научной и коммерческой ценностью.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] ISO 31030:2021. Управление рисками в путешествиях — Руководство для организаций [Электронный ресурс]. — Режим доступа : <https://www.iso.org/standard/54204.html>. — Дата доступа: 22.03.2026.
- [2] GDELT Project [Электронный ресурс]. — Режим доступа : <https://www.gdeltproject.org/>. — Дата доступа: 22.03.2026.
- [3] Lingua Language Detector [Электронный ресурс]. — Режим доступа : <https://github.com/pemistahl/lingua-py>. — Дата доступа: 26.04.2026.
- [4] fastText language identification [Электронный ресурс]. — Режим доступа : <https://fasttext.cc/docs/en/language-identification.html>. — Дата доступа: 26.04.2026.
- [5] No Language Left Behind: Scaling Human-Centered Machine Translation [Электронный ресурс]. — Режим доступа : <https://arxiv.org/abs/2207.04672>. — Дата доступа: 26.04.2026.
- [6] Beyond English-Centric Multilingual Machine Translation [Электронный ресурс]. — Режим доступа : <https://arxiv.org/abs/2010.11125>. — Дата доступа: 26.04.2026.
- [7] Marian: Fast Neural Machine Translation in C++ [Электронный ресурс]. — Режим доступа : <https://arxiv.org/abs/1804.00344>. — Дата доступа: 26.04.2026.
- [8] OPUS-MT: Building open translation services for the World [Электронный ресурс]. — Режим доступа : <https://aclanthology.org/2020.eamt-1.61/>. — Дата доступа: 26.04.2026.
- [9] Qwen2.5: A Party of Foundation Models [Электронный ресурс]. — Режим доступа : <https://qwenlm.github.io/blog/qwen2.5/>. — Дата доступа: 26.04.2026.
- [10] Mistral 7B [Электронный ресурс]. — Режим доступа : <https://arxiv.org/abs/2310.06825>. — Дата доступа: 26.04.2026.
- [11] TextRank: Bringing Order into Texts [Электронный ресурс]. — Режим доступа : <https://aclanthology.org/W04-3252/>. — Дата доступа: 26.04.2026.
- [12] LexRank: Graph-based Lexical Centrality as Salience in Text Summarization [Электронный ресурс]. — Режим доступа : <https://arxiv.org/abs/1109.2128>. — Дата доступа: 26.04.2026.

- [13] Generic Text Summarization Using Relevance Measure and Latent Semantic Analysis [Электронный ресурс]. — Режим доступа : <https://dl.acm.org/doi/10.1145/383952.384042>. — Дата доступа: 26.04.2026.
- [14] BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension [Электронный ресурс]. — Режим доступа : <https://arxiv.org/abs/1910.13461>. — Дата доступа: 26.04.2026.
- [15] PEGASUS: Pre-training with Extracted Gap-sentences for Abstractive Summarization [Электронный ресурс]. — Режим доступа : <https://arxiv.org/abs/1912.08777>. — Дата доступа: 26.04.2026.
- [16] Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer [Электронный ресурс]. — Режим доступа : <https://arxiv.org/abs/1910.10683>. — Дата доступа: 26.04.2026.
- [17] mT5: A massively multilingual pre-trained text-to-text transformer [Электронный ресурс]. — Режим доступа : <https://arxiv.org/abs/2010.11934>. — Дата доступа: 26.04.2026.
- [18] Multilingual Denoising Pre-training for Neural Machine Translation [Электронный ресурс]. — Режим доступа : <https://arxiv.org/abs/2001.08210>. — Дата доступа: 26.04.2026.
- [19] LongT5: Efficient Text-To-Text Transformer for Long Sequences [Электронный ресурс]. — Режим доступа : <https://arxiv.org/abs/2112.07916>. — Дата доступа: 26.04.2026.
- [20] Hugging Face, модель dsliim/bert-base-NER model [Электронный ресурс]. — Режим доступа : <https://huggingface.co/dsliim/bert-base-NER>. — Дата доступа: 22.03.2026.
- [21] Полнотекстовый геопарсинг Mordecai 3 [Электронный ресурс]. — Режим доступа : <https://github.com/openeventdata/mordecai>. — Дата доступа: 22.03.2026.
- [22] Hugging Face, модель MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli [Электронный ресурс]. — Режим доступа : <https://huggingface.co/MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli>. — Дата доступа: 22.03.2026.
- [23] Сервис Ollama [Электронный ресурс]. — Режим доступа : <https://ollama.com/>. — Дата доступа: 22.03.2026.
- [24] Язык программирования Java [Электронный ресурс]. — Режим доступа : <https://www.java.com/en/>. — Дата доступа: 22.03.2026.

- [25] Документация по фреймворку Spring [Электронный ресурс]. — Режим доступа : <https://spring.io/projects/spring-framework>. — Дата доступа: 22.03.2026.
- [26] Документация по фреймворку Spring Boot [Электронный ресурс]. — Режим доступа : <https://spring.io/projects/spring-boot>. — Дата доступа: 22.03.2026.
- [27] Язык программирования Python [Электронный ресурс]. — Режим доступа : <https://www.python.org/>. — Дата доступа: 22.03.2026.
- [28] Документация по Python 3.11 [Электронный ресурс]. — Режим доступа : <https://docs.python.org/3.11/>. — Дата доступа: 22.03.2026.
- [29] Документация по фреймворку FastAPI [Электронный ресурс]. — Режим доступа : <https://fastapi.tiangolo.com/>. — Дата доступа: 22.03.2026.
- [30] Библиотека PyTorch для глубокого обучения [Электронный ресурс]. — Режим доступа : <https://pytorch.org/>. — Дата доступа: 22.03.2026.
- [31] Документация по СУБД PostgreSQL [Электронный ресурс]. — Режим доступа : <https://www.postgresql.org/docs/>. — Дата доступа: 22.03.2026.
- [32] Пространственное расширение PostGIS [Электронный ресурс]. — Режим доступа : <https://postgis.net/>. — Дата доступа: 22.03.2026.
- [33] Брокер сообщений RabbitMQ [Электронный ресурс]. — Режим доступа : <https://www.rabbitmq.com/>. — Дата доступа: 22.03.2026.
- [34] Документация по RabbitMQ [Электронный ресурс]. — Режим доступа : <https://www.rabbitmq.com/documentation.html>. — Дата доступа: 22.03.2026.
- [35] Библиотека React для создания интерфейсов [Электронный ресурс]. — Режим доступа : <https://react.dev/>. — Дата доступа: 22.03.2026.
- [36] Документация по библиотеке React [Электронный ресурс]. — Режим доступа : <https://react.dev/>. — Дата доступа: 22.03.2026.
- [37] Библиотека MapLibre GL JS для визуализации карт [Электронный ресурс]. — Режим доступа : <https://maplibre.org/>. — Дата доступа: 22.03.2026.
- [38] Документация по библиотеке MapLibre GL [Электронный ресурс]. — Режим доступа : <https://maplibre.org/maplibre-gl-js/docs/>. — Дата доступа: 22.03.2026.

[39] Национальный статистический комитет Республики Беларусь [Электронный ресурс]. — Режим доступа : <https://www.belstat.gov.by/ofitsialnaya-statistika/solialno-demograficheskaya-statistika/trud/>. — Дата доступа: 22.03.2026.